

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Subagent: delegare o fare da sé

Che cos’è un subagent

Un contesto nuovo, un compito, e alla conversazione torna il risultato

- Nel Modulo 4, dopo i permessi e la rete di sicurezza: Claude Code può passare un pezzo di lavoro a un **subagent**. Che cos’è, quando conviene, e quanto costa
- Un subagent lavora in un **contesto suo**, con un suo prompt di sistema, i suoi strumenti e i suoi permessi. Parte **nuovo e isolato**: non vede la vostra conversazione, né i file che Claude ha già letto. Fa eccezione un **fork**, che eredita tutta la conversazione: Claude può avviarne uno da sé quando la fork mode è attiva, e lo è di serie
- Quando un compito corrisponde alla **descrizione** di un subagent, Claude gli delega il lavoro: il subagent lavora da solo e restituisce i risultati. La conversazione principale non vede le sue chiamate agli strumenti né i loro output intermedi: **solo il risultato finale**
- Avviarlo non chiede permesso; ma le **sue chiamate** agli strumenti Claude Code le controlla con le vostre regole di permesso, mentre lavora
- Senza configurazione, un subagent avviato dalla conversazione principale parte **in background**: Claude continua a lavorare mentre gira

Il tool Agent, e quelli già pronti

Un nome nuovo, uno vecchio, e tre subagent di serie

- Il subagent lo avvia il tool **Agent**, con una sua finestra di contesto. Prima della versione 2.1.63 si chiamava **Task**; e nello stream di `claude -p` si vedono tutti e due: `Agent` nelle chiamate, `Task` nell'elenco degli strumenti dell'avvio
- **Explore**: solo strumenti di lettura, Write ed Edit negati; il modello della conversazione principale (se questa usa Fable, dipende da come vi collegate); parte senza i vostri `CLAUDE.md`. Le ricerche ampie nella codebase Claude le manda a Explore **da sé**
- **Plan**: la ricerca di plan mode, che raccoglie il contesto prima di presentare un piano
- **general-purpose**: compiti complessi, in più passi, che chiedono di esplorare e di agire, con tutti gli strumenti dei subagent

I vostri: `.claude/agents/`

Un file per subagent: in testa chi è, sotto il suo prompt

Il **revisore** del laboratorio, un subagent in sola lettura:

`revisore.md del laboratorio: «[…]» segna un taglio`

```
---
name: revisore
description: Rilegge le modifiche […] Non modifica niente.
tools: Read, Grep, Glob
model: haiku
---
[…] Non hai visto
scrivere il codice: lo guardi da fuori, e non modifichi niente.
[…]
Rispondi in questo formato, e con nient'altro:

ESITO: OK | DA CORREGGERE
REQUISITI:
- R<n>: soddisfatto | parziale | mancante — <una riga> — […]
PROBLEMI:
- <file>:<riga> — <che cosa non va> — <requisito o regola>
(«- nessuno» se non ce ne sono)
```

I vostri: `.claude/agents/`

Un file per subagent: in testa chi è, sotto il suo prompt

- Di **progetto** in `.claude/agents/`, per una codebase; **vostri** in `~/.claude/agents/`, per tutti i progetti. Se la cartella non c'era all'avvio della sessione, Claude Code va **riavviato**
- Obbligatori solo `name` e `description`. Un campo che Claude Code non riconosce lo ignora, **senza errori**: attenti ai nomi dei campi
- `tools` elenca gli strumenti consentiti, `disallowedTools` quelli negati. Con soli Read, Grep e Glob il revisore **non può modificare nulla**: lo impone Claude Code, non il prompt
- `model: haiku`, se il lavoro è semplice
- «use proactively» nella descrizione incoraggia Claude a delegare. Nominare il subagent nel prompt lascia a Claude la scelta; la **@-mention** garantisce che parta
- E un **formato di risposta** nel suo prompt: per l'Academy di Anthropic, il miglioramento più importante che si possa fare a un subagent

Quando ripagano, e quanto costano

La pagina delle best practice di Claude Code, nella versione attuale

- Fra i consigli che la pagina dà ormai per **superati**: «Use subagents to protect your context window». Perché: «Running out of context is rarely the problem anymore.»
- Restano utili per il lavoro **in parallelo**, per tenere lontano l'output **rumoroso** e per una **revisione indipendente**; ma ognuna di queste cose costa token in più
- Quando ripagano: lavori indipendenti nello **stesso momento**; un **revisore** che non ha visto scrivere il codice; un **output lungo**, come un'intera esecuzione dei test o una ricerca ampia, tenuto fuori dalla conversazione principale; un compito con **meno strumenti** della sessione principale, come un revisore che legge e non modifica
- Che cosa costano: soprattutto **token**, perché ogni subagent manda le sue richieste, e contano sullo stesso utilizzo della conversazione principale; a volte **tempo**: un subagent nuovo deve prima raccogliere il contesto che la conversazione aveva già
- Che cosa se ne ottiene: parallelismo, o uno sguardo fresco, **non una risposta più intelligente**. Passargli lo stesso compito non migliora la risposta; un revisore fresco, che non ha visto il vostro ragionamento, sì
- La domanda dell'Academy, per decidere: il **lavoro intermedio** conta? E si resta nella conversazione principale quando servono **molti scambi**, o quando più fasi condividono molto contesto

M4-D06 Delegare la lettura

Demo del docente in Claude Code: la stessa domanda, con e senza un subagent

- Due esecuzioni che differiscono in **una cosa sola**: la prima frase del prompt. La stessa domanda su una **settimana di log** della rete del corso (syslog, PPPoE, RADIUS, DHCP, SIP), con la delega a **Explore** e senza

`prompt_delega.txt`, la coppia con la delega

```
Delega la lettura dei log a un subagent Explore e fatti riportare
solo un riassunto breve. Martedì 8 settembre 2026 un guasto su
olt-pergine-01 ha messo fuori servizio insieme più linee di
clienti. Quando è cominciato, quando è finito (quando le ONT sono
tornate in linea) e quante linee di clienti (ONT) ha coinvolto?
Rispondi in una riga: 'inizio: HH:MM | fine: HH:MM | ONT: N'.
```

- `prompt_senza.txt` è uguale, tranne la prima frase: «Non delegare a subagent: leggi tu i log.»
- `claude -p` parte da `progetti/rete/log/`, dove ci sono soltanto i log: Sonnet, effort low, dontAsk; come strumenti soltanto Agent, Read, Grep e Glob, **niente Bash**; lo `stream-json` registra anche le chiamate del subagent agli strumenti, con i loro risultati
- Guardate che cosa arriva alla **conversazione principale**, quanto **contesto** le resta all'ultima richiesta, e quanti **token** costa il tutto

M4-D06 Delegare la lettura

Demo del docente in Claude Code: la stessa domanda, con e senza un subagent

- Dal riferimento degli strumenti di Claude Code: «The parent doesn’t see the subagent’s intermediate tool calls or outputs, only that final result.» Una **garanzia** del prodotto: deve valere in ogni esecuzione con la delega
- Dalla pagina dei subagent, quando usarne uno: «Use one when a side task would flood your main conversation with search results, logs, or file contents you won’t reference again». Dalla pagina delle best practice: «Keeping long output, like a full test run or a wide search, out of your main conversation». Un **confronto**: si decide coppia per coppia
- E il prezzo, nella stessa pagina: «What they cost you is mostly tokens, since every subagent sends its own requests and they count against the same usage as your main conversation.» «You get parallelism or a fresh pair of eyes out of it, but not a smarter answer.»
- Il controllo dell’elemento legge gli stream di ogni coppia: chi chiama quali strumenti; il **contesto principale** all’ultima richiesta; i token di **tutti i modelli**, dal `modelUsage`, che conta l’albero intero; e le risposte confrontate con la verità, come **dati**

M4-D06 Delegare la lettura

Demo del docente in Claude Code: la stessa domanda, con e senza un subagent

Indovinate, a voce: con il subagent, il contesto principale sarà più grande o più piccolo? E il conto totale?

	contesto principale, all'ultima richiesta		tutti i modelli, token in entrata	
coppia	con la delega	senza	con la delega	senza
1	9.948	16.318	66.282	42.108
2	10.014	16.150	66.543	32.931
3	9.781	16.318	55.471	42.108
in tutto			188.296	117.147

Tre coppie, ogni sessione nuova: i valori che il controllo ha stampato, in tabella
Registrazione del corso, 28 ottobre 2026, Claude Code 2.1.312, Sonnet

- La garanzia: «La conversazione principale vede solo il risultato del subagent: **confermata 3 su 3**», con 3 risultati di strumenti del subagent in ogni esecuzione, nessuno nella conversazione principale
- Il confronto: «Un subagent tiene i log fuori dalla conversazione principale: **confermata 3 su 3**»
- I dati: «tutti i modelli: 188.296 token in entrata con la delega, 117.147 senza; risposte giuste: 3 su 3 con la delega, 3 su 3 senza»
- La risposta: il contesto principale più **piccolo**; il conto più **grande**; e la stessa risposta giusta, da una parte e dall'altra

Delegare bene

Poco in uscita dal subagent, e il conto sotto gli occhi

- Chiedete **riassunti brevi**: «Ask for short summaries back, so twenty results don’t fill your context anyway.» Il prompt della demo lo chiedeva
- `model: haiku` per i lavori **semplici**
- Un **formato di risposta** nel prompt del subagent, come il revisore del laboratorio
- Sul Pro, `/usage` divide l’utilizzo per skill, **subagent**, plugin e server MCP: guardatelo dopo una sessione con i subagent

Un subagent legge al posto vostro e vi riporta un riassunto: il contesto principale resta piccolo, i token si pagano lo stesso, e la risposta non è più intelligente.